

Carbon-Flow: Autonomous Decarbonization Platform

1. Project Overview

Carbon-Flow is a zero-friction, AI-driven sustainability dashboard designed to bridge the "Value-Action Gap." Traditional carbon trackers fail because of the heavy manual data entry required. Carbon-Flow acts as a **smart, dynamic assistant** that uses multimodal AI to ingest raw user data (receipts, utility bills, voice logs), calculate hyper-localized carbon equivalents, and drive behavioral change through a mathematically optimized gamification engine.

2. Core Mechanics (Addressing Challenge Expectations)

- **Practical & Real-World Usability (Zero-Friction Tracking):** Users simply snap a photo of a utility bill or record a quick voice note. The Python ingestion layer uses OpenCV for preprocessing and the Gemini API for direct vision-to-token data extraction, eliminating manual entry.
- **Logical Decision Making based on Context:** The Java Spring Boot backend doesn't use generic math. It applies regional emission factors (like EPA eGRID coefficients) based on the user's localized context to ensure absolute accuracy in carbon tracking.
- **Dynamic Assistant (Behavioral Decarbonization):** Instead of static push notifications, the platform uses a Markov Decision Process (MDP) to calculate the user's "Habit Strength." The React UI—designed with a dynamic, Pixar-meets-cyberpunk aesthetic—visually rewards consistency (glowing neon oasis) and penalizes omission (dark, glitching industrial UI).

3. Evaluation Focus Areas

Security & Privacy

- **Implementation:** All data exchanges between the React frontend and Java API gateway are secured via JWT authentication.
- **Validation:** The Python layer enforces strict schema generation using `PydanticAI`, and the Java backend utilizes `networknt/json-schema-validator` to ensure all payloads are sanitized before database insertion, preventing injection attacks.

Efficiency & Optimal Resource Use

- **Token Economics:** Document processing routes standard layouts through the lighter, cost-effective Gemini 2.5 Flash-Lite, only cascading to Gemini 2.5 Pro for highly complex, multi-page utility bills.
- **Payload Optimization:** Images are locally compressed via Pillow and preprocessed with adaptive contrast thresholding to minimize network latency and API token burn.

Code Quality & Architecture

- **Maintainable Separation of Concerns:** A strict Full-Stack Ironclad Architecture isolates AI routing logic (Python/FastAPI) from heavy business logic and transaction management (Java)

Spring Boot 3).

- **Documentation:** Clean, modular functions with comprehensive docstrings mapping out the decoupled asynchronous task queues (Celery/RabbitMQ) that prevent API blocking during image uploads.

Testing

- **Backend Validation:** Comprehensive unit testing of the emission calculation formulas and the MDP gamification math using JUnit (Java) and PyTest (Python). Mocked API calls ensure the AI ingestion layer degrades gracefully during rate limits.

Accessibility

- **Inclusive Design:** The highly visual Tailwind dashboard maintains strict WCAG contrast ratios. All dynamic animations respect the `prefers-reduced-motion` CSS media query, ensuring the UI is safe and usable for everyone.

4. Submission & Deployment Strategy (Critical Execution Plan)

The hackathon's **Score Multiplier Guidelines** completely dictate our deployment workflow. Because a lower base score submitted early can outrank a late perfect score, we must operate with ruthless efficiency.

The Workflow Rules:

1. **Single Branch Protocol:** All prompting, autonomous agent generation, and manual commits will live purely on the `main` branch of the public GitHub repository. No complex merging strategies; we keep it linear and fast.
2. **MVP First, Polish Later:** We will push a functional, mathematically sound backend and a basic frontend as our **Attempt 1** the moment it validates. This locks in a high time-dependent multiplier. Attempts 2 and 3 will be used for UI polish and edge-case patching.
3. **Deployment Watchdog:** When pushing the repository to the AI platform for live deployment, monitor the pipeline closely. If the deployment stalls, cancel and restart immediately—**build processes frequently get stuck specifically on "uploading sources" rather than the container build itself**. Do not let a hung upload eat into our time multiplier.
4. **Regular Commits:** Commit every time an autonomous agent successfully completes one of the 4 micro-phases (Ingestion, Java Gateway, Gamification Math, Frontend).